LabBench
Interview Prep Kit

Common ECE interview questions on DSP, Control Systems,
Digital Logic, and Communication Systems — with worked answers.

by Dhananjay Kumar Seth
labbench-hub.vercel.app

How to use this kit

These are the questions that come up again and again in ECE fresher/entry-level interviews and viva exams — not exhaustive theory, but the specific things interviewers reach for to check you actually understand a concept rather than having memorized it. Each answer is written the way you'd actually say it out loud: concise, correct, and with the reasoning visible.

Where it helps, an answer points back to a LabBench tool — being able to say "I can show you" and pull up a live demo is a stronger signal than reciting a definition.

Contents

1. Digital Signal Processing

Q1. What is the Nyquist–Shannon sampling theorem, and what happens if you violate it?

It states that to perfectly reconstruct a continuous signal from its samples, the sampling rate must be at least twice the highest frequency component in the signal (f_sample >= 2 × f_max). Violating it causes **aliasing** — frequencies above half the sample rate (the Nyquist frequency) fold back and appear as false lower frequencies in the sampled signal, and this corruption cannot be undone after the fact. It's why audio is sampled at 44.1kHz (roughly double the ~20kHz upper limit of human hearing) and why cameras use anti-aliasing filters.

Q2. What's the difference between the time domain and the frequency domain?

The time domain describes a signal as amplitude vs. time — how it looks on an oscilloscope. The frequency domain describes the same signal as a set of frequency components and their amplitudes — which sine waves, added together, would reconstruct it. The Fourier Transform converts between the two; the FFT (Fast Fourier Transform) is just an efficient algorithm for computing it on a computer. Neither view contains more information than the other — they're two representations of the same signal, and you pick whichever makes the problem you're solving easier to see.

> ***Interview tip:*** *If asked to prove you understand this rather than just define it: a square wave is a strong example. In the time domain it's just two levels alternating; in the frequency domain it's an infinite sum of odd harmonics at decreasing amplitude. Being able to explain why gives a much stronger answer than the definition alone.*

Q3. Explain the difference between an FIR and an IIR filter.

An FIR (Finite Impulse Response) filter's output depends only on a finite window of current and past input samples — no feedback. It's always stable and can have exactly linear phase, but usually needs more taps (more computation) for a sharp cutoff. An IIR (Infinite Impulse Response) filter feeds its own past output back into the calculation, which lets it achieve a much sharper response with far fewer coefficients, but it can become unstable if designed carelessly, and generally has non-linear phase. A BiquadFilterNode (as used in DSP Signal Lab) is a common, well-behaved IIR filter building block used in real audio software.

Q4. What is a filter's Q factor, and how does it relate to bandwidth?

Q (quality factor) is the ratio of a filter's center/cutoff frequency to its bandwidth: Q = f_c / bandwidth. A high Q means a narrow, sharply resonant response — useful for isolating one specific frequency, but more sensitive to component tolerances in a real circuit. A low Q means a broad, gentle roll-off. In a notch filter specifically, high Q means it removes only a very narrow band (e.g. exactly 50Hz mains hum) without touching nearby frequencies.

Q5. Why is white noise called 'white,' and how is it different from other noise types?

White noise contains equal energy at every frequency, the same way white light contains all visible wavelengths equally — hence the name. Pink noise, by contrast, has energy that falls off with increasing frequency (equal energy per octave, not per Hz), which is closer to how many natural

and electronic noise sources actually behave. In DSP work, white noise is the standard test signal because its flat spectrum makes it easy to characterize a filter's response — feed it in and whatever comes out directly shows the filter's frequency response.

Interview tip: *Try it: open DSP Signal Lab, mix in some noise, and sweep a bandpass filter across the spectrum — you're literally watching a filter's frequency response get traced out by the noise.*

2. Control Systems (PID)

Q1. Explain what each term in a PID controller does, in plain language.

Proportional (P) reacts to how large the current error is — bigger error, bigger correction, but alone it always leaves some steady-state offset and tends to oscillate. Integral (I) reacts to the accumulated error over time — it's what eliminates that steady-state offset, since even a tiny persistent error keeps building the integral until it forces a correction, but too much I causes slow overshoot ('integral windup'). Derivative (D) reacts to how fast the error is changing — it acts as a brake, pulling back the output when the error is closing quickly to prevent overshoot, adding damping, but amplifying noise if set too high.

Q2. What is integral windup, and how is it usually prevented?

Integral windup happens when the integral term keeps accumulating error while the system is saturated (physically unable to respond further — e.g. a motor already at full power), building up a huge integral value. When the system finally becomes able to respond again, that oversized integral causes a large overshoot before it can unwind. The standard fix is **clamping**: cap the integral term (or the controller's total output) at a sane maximum so it can never accumulate beyond what the system could physically use.

Q3. How would you tune a PID controller from scratch?

A common manual approach (close to the classic Ziegler–Nichols method): start with only P, increasing Kp until the system oscillates steadily at a constant amplitude — note that gain and the oscillation period. Back off Kp somewhat from that point, then add D to damp overshoot, and finally add a small amount of I to eliminate any remaining steady-state error, increasing it just enough to close the gap without reintroducing slow oscillation. In practice, tuning is always a trade-off: faster response (higher Kp) tends to fight against overshoot control (Kd) and against eliminating steady-state error (Ki) without windup.

> ***Interview tip:*** *PID Control Playground has this trade-off built in as presets — loading 'P-only' then 'PD' then 'PID' in sequence is a fast way to visually demonstrate you understand what each term is fixing.*

Q4. What's the difference between open-loop and closed-loop control?

Open-loop control applies a predetermined action without checking the actual output — a microwave running for a fixed time regardless of whether the food is actually hot. Closed-loop (feedback) control continuously measures the actual output and adjusts the input based on the error between target and actual — a thermostat that keeps checking room temperature and adjusting heating accordingly. PID control is closed-loop by definition: it cannot function without a feedback measurement of the current error.

Q5. What do rise time, settling time, and steady-state error each measure?

Rise time is how quickly the output first reaches the target after a setpoint change. Settling time is how long until the output stays within a small tolerance band around the target (accounting for any overshoot/ringing along the way). Steady-state error is the gap that remains indefinitely once the system has settled — ideally zero for a well-tuned PID with sufficient integral action. These three, together with overshoot percentage, are the standard vocabulary for describing any step response,

in ECE coursework and in real control system datasheets alike.

3. Digital Logic & Sequential Circuits

Q1. What's the difference between combinational and sequential logic?

A combinational circuit's output depends only on its current inputs — feed it the same inputs twice and you always get the same output, with no memory involved (AND/OR gates, adders, multiplexers). A sequential circuit's output depends on current inputs *and* its history, because it has memory (flip-flops, counters, shift registers, state machines). The presence of feedback — an output looping back to influence a future input — is what gives a circuit memory.

Q2. Explain edge-triggered vs. level-triggered behavior in a flip-flop/latch.

A level-triggered latch (like a basic SR or gated D latch) is 'transparent' whenever its enable/clock input is at the active level — its output can change any time the input changes while enabled, which makes timing harder to reason about in a larger synchronous design. An edge-triggered flip-flop only captures its input at the instant of a clock transition (e.g. the rising edge) and holds that value steady until the next edge, completely ignoring input changes in between. Edge-triggering is what makes large synchronous digital systems (CPUs, counters, pipelines) predictable — every register updates at exactly the same well-defined moment.

> ***Interview tip:*** *Waveform Viewer makes this literal: click a cell in the D row, and Q only changes exactly one clock period later, at the edge — not immediately. That one-period delay is edge-triggering, made visible.*

Q3. How would you design a circuit that divides a clock frequency by 2? By 8?

A single T flip-flop (or a D flip-flop with its inverted output wired back to D) toggles on every rising edge, which halves the clock frequency — its output period is exactly twice the input clock's. Chaining N such divide-by-2 stages, each one clocked by the previous stage's output, divides the frequency by 2^N — so three chained stages divide by 8. This ripple-chain pattern is also exactly how a binary ripple counter's bits behave: bit 0 toggles every cycle, bit 1 every 2 cycles, bit 2 every 4.

Q4. What is a race condition / hazard in a digital circuit, and how do latches with feedback create one?

A hazard is a temporary, unwanted glitch in a circuit's output caused by differing propagation delays along different paths to the same point, even though the final settled value is correct. In a circuit with feedback (like an SR latch built from two cross-coupled NOR gates), there's no valid single left-to-right evaluation order — each gate's output depends on the other's, so a naive evaluator can deadlock or need to be evaluated iteratively (repeatedly re-checking every gate until nothing changes any more) to reach a stable state, which is exactly how real hardware settles too, just far faster.

Q5. Design a 2-to-1 multiplexer's truth table and Boolean expression.

A 2-to-1 MUX has inputs A, B, a select line S, and output Y, where Y = A when S=0 and Y = B when S=1. As a Boolean expression: Y = (NOT S AND A) OR (S AND B). Every combinational circuit reduces to exactly this kind of input-to-output mapping — knowing you can always fall back to 'just write the truth table, then derive the expression' is a safe way to answer any 'design a circuit that does X' question under pressure.

4. Communication Systems

Q1. Why do we modulate a signal instead of transmitting it directly?

Baseband signals like voice (roughly 300Hz-3kHz) can't be radiated efficiently — an antenna needs to be a meaningful fraction of the signal's wavelength, and at those low frequencies that wavelength is enormous. Modulation shifts the information onto a much higher-frequency carrier wave, which can be radiated efficiently by a practically-sized antenna and separated from other transmissions by frequency, so many signals can share the airwaves without colliding.

Q2. What's the practical difference between AM and FM, and why does FM sound cleaner?

AM varies the carrier's amplitude in proportion to the message while holding frequency constant; FM varies the carrier's frequency in proportion to the message while holding amplitude constant. Most real-world interference (lightning, electrical noise, weak signal fading) shows up as amplitude fluctuations. Because FM doesn't encode information in amplitude at all, an FM receiver can simply ignore those amplitude variations — which is exactly why FM radio sounds cleaner than AM in the same noisy environment.

Q3. What is a constellation diagram, and what does it tell you at a glance?

A constellation diagram plots each possible transmitted symbol as a point on an I/Q (in-phase / quadrature) plane — the symbol's amplitude and phase encoded as a 2D position. Received symbols, corrupted by channel noise, scatter as a cloud around each ideal point instead of landing exactly on it. At a glance, tighter clouds mean higher SNR (cleaner channel); clouds that overlap between adjacent ideal points mean the receiver will make decision errors — literally the source of bit errors, made visible.

> ***Interview tip:*** *Interviewers often ask you to compare modulation orders directly: '16-QAM packs 4 bits per symbol vs. BPSK's 1, but its constellation points are packed closer together, so it needs meaningfully higher SNR for the same error rate' is the concise, correct answer to almost any bit-rate-vs-noise question in this space.*

Q4. What is BER, and what does it mean for a BER curve to match theory?

Bit Error Rate is the fraction of received bits that come out wrong after demodulation. A theoretical BER curve (derived from the Gaussian Q-function for schemes like BPSK) predicts error rate as a function of SNR under ideal assumptions. A real or simulated system's measured BER curve matching that theoretical curve closely is one of the standard ways engineers confirm a communication link's receiver is implemented correctly — a real system will never beat the theoretical curve, only approach it, so any curve suspiciously better than theory usually signals a measurement bug rather than a genuinely better receiver.

Q5. Why does higher-order modulation (like 16-QAM vs. BPSK) need a higher SNR for the same error rate?

For a fixed transmit power, packing more bits into each symbol (higher-order modulation) means spreading the same total signal energy across more constellation points, packed into the same region of the I/Q plane — so the distance between adjacent points shrinks. Since noise displaces a

received symbol randomly in that plane, closer-together ideal points are far easier to mistake for each other under the same amount of noise. That's the fundamental bit-rate-vs-robustness trade-off present in every real wireless standard: higher throughput per symbol always costs SNR margin.